Sales Forecasting

Fresh Analytics — Predicting Item Demand Across Restaurants (2019–2021)

Project Summary & Interview Preparation Guide

Author: Ajaykanna E | Capstone Project

1. Project Overview

Fresh Analytics operates a network of restaurants and needs to accurately forecast daily item demand. Accurate forecasting helps the business plan inventory, schedule staff, and run targeted promotions. This project builds a complete data science pipeline — from raw CSV data through exploratory analysis to machine learning and deep learning models — culminating in a 90-day sales forecast for 2022.

Business Problem

- Demand varies heavily by day of week, season, and item type — making manual estimation unreliable.
- Under-ordering leads to stockouts. Over-ordering leads to food waste and lost profit.
- The goal: build models that predict daily item counts so the business can plan ahead with confidence.

Datasets Used

File	Columns	Description
sales.csv	date, item_id, item_count, price	Daily sales records per item per restaurant
items.csv	id, store_id, name, kcal, cost	Item details — name, calories, cost
restaurants.csv	id, name	Restaurant names and IDs

Full Project Pipeline

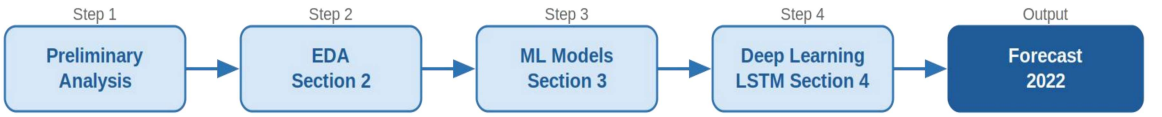

Figure 1 — Complete workflow from raw data to 2022 forecast

2. Section 1 — Preliminary Analysis

Before any modeling, we need to understand what data we have, confirm its quality, and prepare a unified working dataset.

1a. Libraries and Why We Use Them

Library	Purpose in This Project
pandas	Load CSVs, merge tables, group and aggregate data, extract date features
numpy	Array operations and math needed for LSTM sequence creation
matplotlib / seaborn	Visualisations — line charts, bar charts, heatmaps, boxplots
sklearn	Linear Regression, Random Forest, MinMaxScaler, train/test split, metrics
xgboost	Gradient boosting model for tabular time-series forecasting
tensorflow / keras	Deep learning framework — build and train the LSTM model

1b. Dataset Structure and Outlier Detection

We check shape, data types, and missing values before doing anything else. The date column starts as a string and must be converted to datetime so Python can extract day-of-week, month, etc.

Observations from Structure Check

- No missing values in any of the three datasets — data is clean and ready.
- The date column is stored as string (object type) and needs datetime conversion.
- The price column is total daily revenue for that item, not a unit price.
- The sales dataset has many rows with item_count = 0 (days when an item was not sold).

IQR Outlier Detection Method

We use the Interquartile Range method to identify statistical outliers. The key step is filtering out zero-sales rows first — if we include them, Q1 = Q3 = 0 and every sale gets flagged as an outlier.

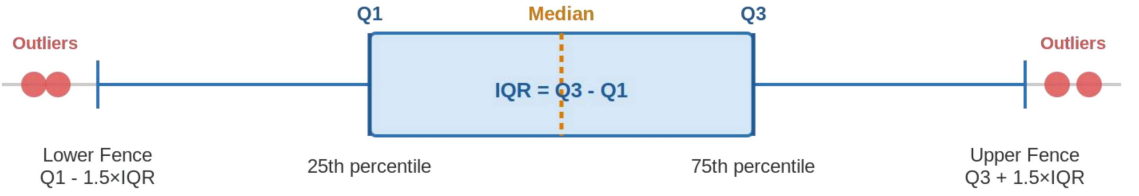

Figure 2 — IQR method: values beyond 1.5 × IQR from Q1/Q3 are flagged as outliers

Why We Keep the Outliers Here

- High-sales outliers are genuine peak demand events — a popular drink on a busy Friday.
- These are exactly the peaks our model needs to predict. Removing them would make our model worse.
- Outlier treatment in this project means understanding outliers, not always deleting them.

1c. Dataset Merge

Three separate CSV files are joined into one unified dataset using shared key columns. This is equivalent to SQL JOIN operations.

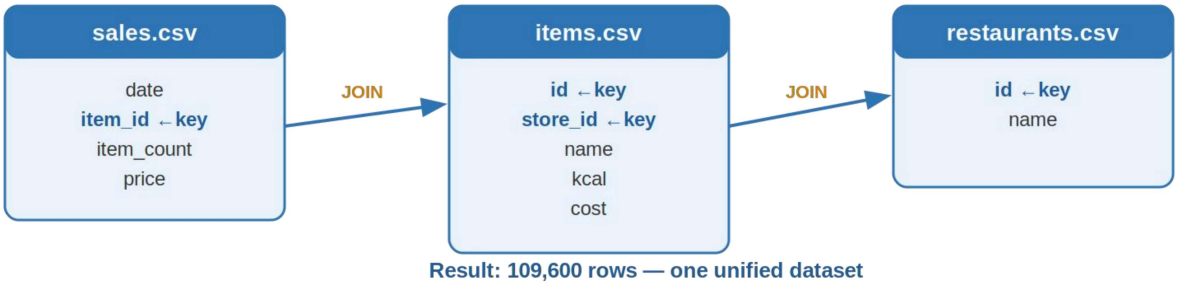

Figure 3 — Three CSVs joined into one 109,600-row unified dataset

Why Merge Before Analysis?

- sales.csv alone doesn't tell us the item name, calories, or restaurant name.
- After merging, every row has full context: date, item name, restaurant, count sold, price, calories.
- No rows lost in the merge — all IDs matched correctly across the three files.

3. Section 2 — Exploratory Data Analysis (EDA)

EDA is how we learn the story inside the data. Good EDA reveals patterns that directly guide modeling decisions — which features to engineer, what the model should prioritize, and what results to expect.

We begin by extracting time-based features from the date column: year, month, quarter, day-of-week, day-of-year, week-of-year, and a binary is_weekend flag.

2a. Overall Sales Trend Over Time

Key Findings — Overall Trend

- Clear weekly repeating pattern throughout all three years — spikes = Fridays, dips = Sundays.
- 7-day rolling average reveals a gradual upward trend year-over-year: the business is growing.
- Seasonal peaks visible mid-year (summer) and dips toward year-end (winter).
- This tells us: time-of-year is a strong predictor and any good model must capture seasonality.

2b. Sales by Day of Week

Day	Pattern	Business Implication
Friday	Highest sales across all 3 years	Peak staffing and inventory day
Saturday	Second highest	Strong weekend start
Sunday	Lowest sales	Customers prefer home meals end of weekend
Monday/Tuesday	Mid-week dip	Typical for casual dining restaurants

This is the single most important finding in EDA. day_of_week becomes the top-ranked feature in both ML models — exactly as expected.

2c. Monthly and Seasonal Trends

Key Findings — Seasonality

- Sales peak from May to August (summer) consistently across all three years.
- January and December show the lowest sales each year — cold weather and post-holiday budgets.
- The seasonal pattern is stable and repeatable — this is a reliable signal models can learn from.
- Year-over-year growth is visible: the same month in 2021 is almost always higher than 2020.

2d. Quarterly Sales Distribution

Quarter	Months	Pattern	Why
Q2	Apr – Jun	Strongest quarter	Spring + early summer dining surge
Q3	Jul – Sep	Second strongest	Peak summer season
Q1	Jan – Mar	Weak	Winter, post-holiday slowdown
Q4	Oct – Dec	Weakest quarter	Holiday season does not boost these restaurants

2e. Restaurant Performance Comparison

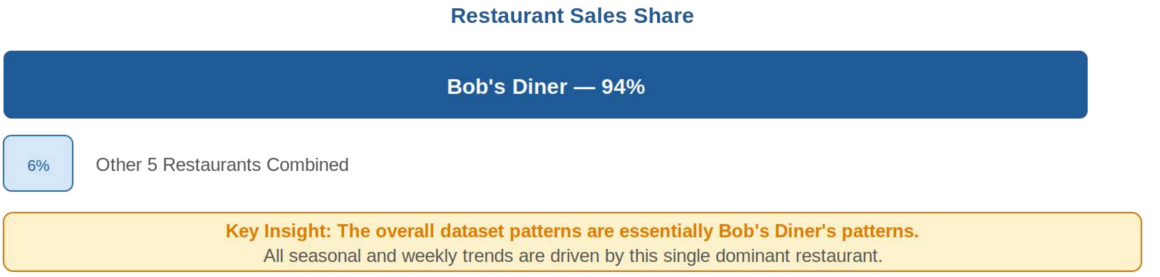

Figure 4 — Bob's Diner accounts for ~94% of all sales in the dataset

Critical Insight — Bob's Diner Dominance

- Bob's Diner: approximately 94% of all sales — a massive imbalance.
- All 6 restaurants follow the same seasonal pattern, so the driver is time-of-year not restaurant-specific factors.
- Bob's Diner peaks on Fridays — confirming it drives the overall weekly pattern seen in Section 2b.
- The other five restaurants contribute so little that aggregate patterns are essentially Bob's Diner's patterns.

2f. Most Popular Items

Rank	Item	Units Sold	Type
1	Strawberry Smoothy	236,000+ units	Drink
2	2nd place item	Roughly half of #1	Drink / Snack
–	Top items overall	Mostly drinks and smoothies	Affordable, quick, repeatable

2g. Volume vs Revenue per Day

Key Finding — Volume and Revenue Aligned

- Bob's Diner leads both total sales volume and average daily revenue.
- High volume translates directly to high daily revenue — no silent revenue leader hiding elsewhere.
- Smaller restaurants earn very little per day despite sometimes having higher-priced items.

2h. Most Expensive Items by Restaurant

Restaurant	Most Expensive Item	Price	Positioning
Fou Cher	Top item	$53.98	Premium fine dining
Bob's Diner	Top item	$29.22	High-volume casual dining
Sweet Shack	Top item	$7.70	Budget-conscious customers

4. Section 3 — Machine Learning Forecasting

Feature Engineering — Why We Can't Use Raw Dates

A date like 2021-06-15 is a string — models have no idea it is summer or a Tuesday. We extract numeric features that encode human intuitions about time.

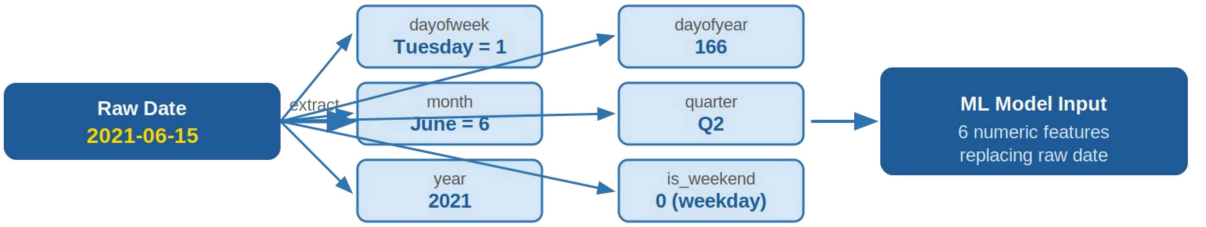

Figure 5 — Feature engineering: raw date broken into 6 numeric inputs for ML models

What Each Feature Captures

- dayofweek — encodes the weekly cycle (Friday = more sales). Top feature in both models.
- month — encodes the seasonal cycle (June = summer peak).
- year — captures the long-term upward growth trend.
- dayofyear — fine-grained seasonal signal (day 1 = Jan 1, day 365 = Dec 31).
- quarter — broad seasonal grouping (Q2 = peak).
- is_weekend — binary signal, but ranks near zero since dayofweek already contains this info.

Train / Test Split — Time-Based

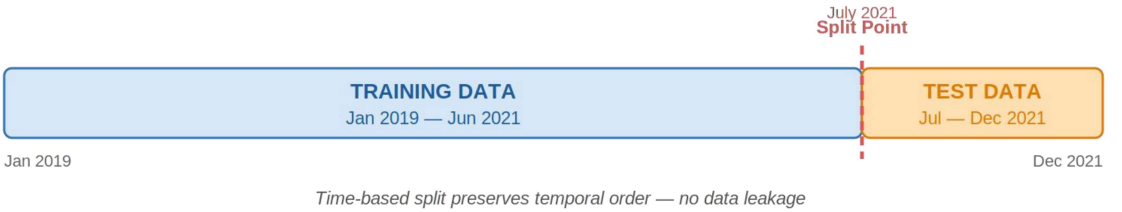

Figure 6 — Time-based split: train on past, evaluate on future — no data leakage

Why We Cannot Split Randomly for Time Series

- Random splitting lets the model see future dates during training — it peeks at what comes next.
- This gives unrealistically good metrics. The model would fail in production.
- Time-based split simulates the real world: train on what you know, predict what comes next.

- Split point: everything before July 2021 = training. July-December 2021 = test.

The Three Models

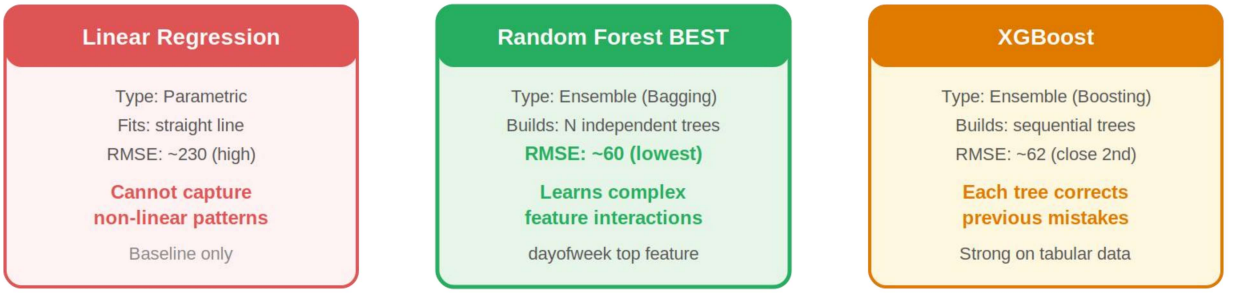

Figure 7 — Three models compared: Linear Regression (baseline), Random Forest (best), XGBoost

How Bagging and Boosting Differ

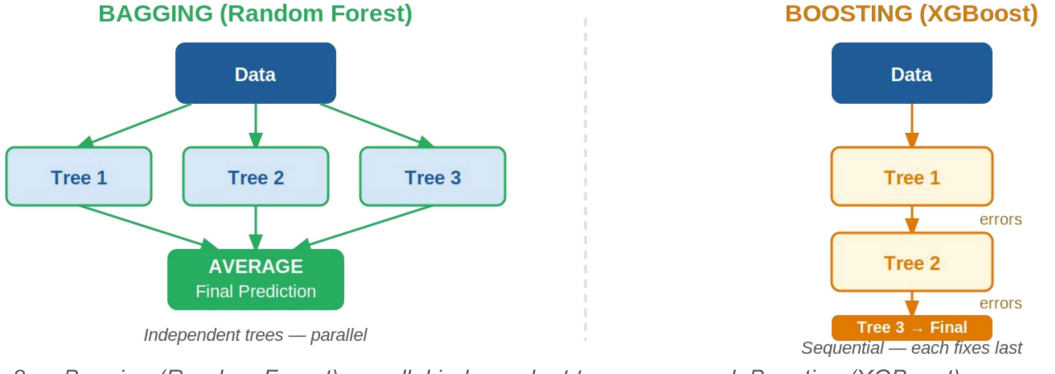

Figure 8 — Bagging (Random Forest): parallel independent trees averaged. Boosting (XGBoost): sequential error-correcting trees

Model Results

Model	RMSE	Performance	Reason
Linear Regression	~230	Poor — baseline only	Cannot capture non-linear seasonal + weekly interactions
Random Forest	~60	Best — winner	Learns complex feature interactions via bagging
XGBoost	~62	Close second	Sequential boosting — strong but similar to RF here

Why RMSE and Not Accuracy?

- This is a regression problem (predicting a number), not classification.
- RMSE = Root Mean Squared Error. It measures average prediction error in the same units as item count.
- RMSE of 60 means predictions are off by ~60 items per day on average.
- RMSE penalizes large errors more than small ones (due to squaring) — appropriate for forecasting.

Feature Importance

Feature	Importance Score	What It Captures
dayofweek	~0.53 (highest)	Weekly cycle — Friday vs Sunday vs weekday
dayofyear	~0.18	Seasonal variation — summer highs, winter lows
weekofyear	Moderate	Broader seasonal grouping
quarter	Moderate	Q2 peak, Q4 low
month	Lower	Somewhat redundant with dayofyear
is_weekend	Near zero	Redundant — dayofweek already captures this

Validation: Feature Importance Matches EDA

- dayofweek scoring 0.53 directly confirms the weekly pattern we found in Section 2b.
- dayofyear as second confirms the seasonal pattern from Section 2c.
- is_weekend near zero confirms that redundant features do not help the model.
- When feature importance matches EDA findings, it is a sign the model learned real structure, not noise.

5. Section 4 — Deep Learning: LSTM Forecasting

While ML models treat each day as an independent observation, the LSTM (Long Short-Term Memory) network treats sales as a temporal sequence — it learns patterns from the order and relationship between days, not just their individual features.

What Is an LSTM and Why Use It for Time Series?

The Core Idea

- Standard neural networks have no memory — they process each input independently.
- LSTM networks have a memory cell that can retain information across many time steps.
- For sales forecasting, this means the model 'remembers' that last week was summer and last Friday was high — and uses that to predict today.
- This temporal memory is exactly what makes LSTM well-suited for sequences like daily sales.

Step 4a to 4c — Data Preparation

Train / Test Split for LSTM

- Same time-based principle as ML models
- Training: 2019 — Jun 2021
- Test: Jul — Dec 2021 (6 months)
- Test set never touched during training

Why 6 Months Test Set?

- 6 months captures both a summer peak and a winter period
- Tests if model generalises across seasons
- Gives stable MAPE estimate across diverse conditions

Step 4d — Synthetic Data for Extended History

Why Generate Synthetic Data?

- Our real data covers Jan 2019 — Dec 2021 (3 years).
- Generating 12 months of synthetic data extending the series gives the LSTM a longer window to learn from.
- The synthetic data preserves: (1) correct seasonal amplitude, (2) weekly cycle, (3) realistic noise level.
- Flat or random synthetic data would break the pattern the LSTM is learning — causing poor predictions.
- Seasonally-aware synthetic data gives the LSTM a seamless, consistent signal — directly improving MAPE.

Step 4e — Normalisation with MinMaxScaler

Why Normalise for LSTM?

- Neural networks are sensitive to the scale of their inputs.

- Sales values range from ~2,000 to ~12,000. Without scaling, gradients become unstable during training.
- MinMaxScaler scales all values to [0, 1] — giving every time step equal numerical standing.
- CRITICAL: Fit the scaler on training data only, then apply the same transformation to test data.
- Fitting on test data = data leakage — the model would 'know' the range of future values.

Step 4f — Sliding Window Sequences

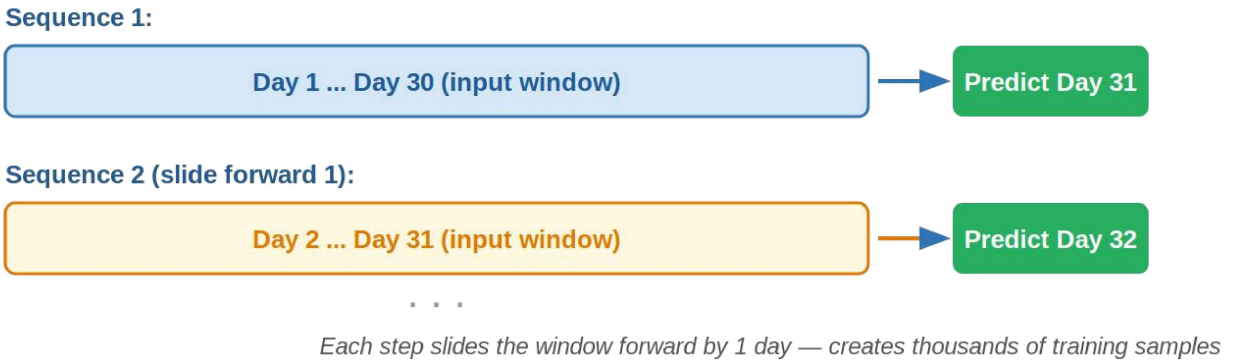

Figure 9 — Sliding window: feed 30 days, predict day 31. Slide forward, repeat

Why 30 Days?

- 30 days captures one full monthly cycle — enough context for the model to understand where it is in the seasonal pattern.
- Shorter windows (e.g. 7 days) might miss the monthly rhythm.
- Longer windows (e.g. 90 days) make training slower and can introduce more noise.
- Each slide creates one (X, y) training pair — so 3 years of data generates thousands of training samples.

Step 4g — LSTM Architecture

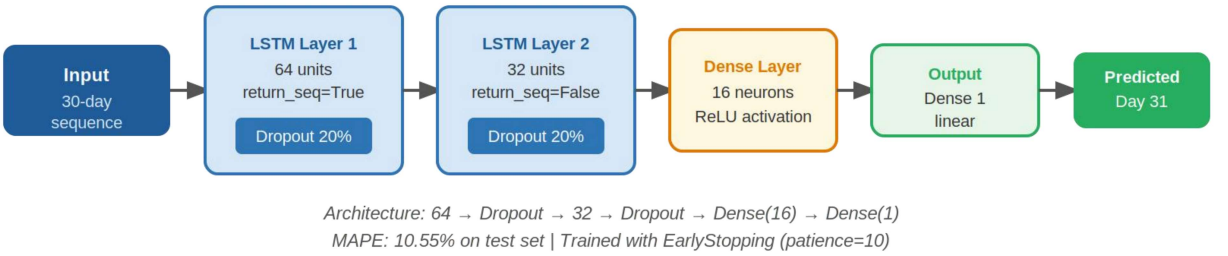

Figure 10 — LSTM architecture: two LSTM layers with Dropout, followed by Dense layers

Layer	Units / Type	Key Setting	Purpose
LSTM Layer 1	64 units	return_sequences=True	Process sequence, pass full output to next layer
Dropout 1	—	rate=0.2	Randomly drops 20% of neurons — prevents overfitting
LSTM Layer 2	32 units	return_sequences=False	Learn higher-level temporal patterns
Dropout 2	—	rate=0.2	Additional regularisation
Dense Layer	16 neurons	ReLU activation	Combine LSTM output into compact representation
Output Layer	1 neuron	Linear (no activation)	Final prediction — one value (next day sales)

Step 4h — Training with EarlyStopping

What EarlyStopping Does

- Monitors validation loss after each training epoch.
- If validation loss does not improve for 10 consecutive epochs (patience=10), training stops.
- Best weights are automatically restored — so we always save the model at its optimal point.
- Without EarlyStopping: training would run all 50 epochs even if the model peaked at epoch 20.
- Continuing past the optimal point = overfitting — model memorises training data, performs worse on new data.

Step 4i and 4j — Predictions and MAPE

After predicting on the test set, we inverse-transform the scaled predictions back to real sales values (undoing MinMaxScaler), then compute MAPE.

Figure 11 — MAPE performance scale. Our LSTM achieves 10.55% — solidly in the Good range

Comment on Model Performance

- LSTM achieves MAPE of 10.55% — falls in the Good performance range (10-20%).
- Given high volatility (daily sales range from $2,000 on quiet winter Sundays to $12,000

on peak summer Fridays), 10.55% error is strong for business planning.

- The predicted curve closely tracks the actual sales curve in shape, magnitude, and weekly rhythm.
- The model has genuinely learned the underlying seasonal and weekly patterns.

Step 4k — Recursive Forecasting for 90 Days

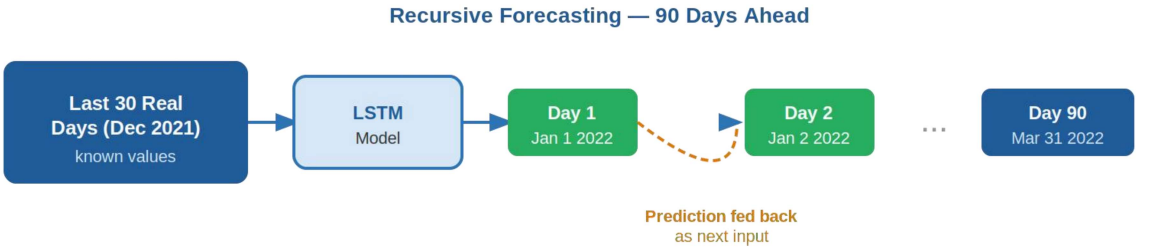

Figure 12 — Recursive forecasting: each predicted day becomes part of the input for the next prediction

Key Observations on the 2022 Forecast

- Forecast correctly picks up from Dec 2021 and immediately resumes the weekly cycle.
- Gradual upward trend from Jan through Mar 2022 — makes sense: January is the winter low, sales recover toward spring.
- Forecast amplitude widens as we move into Feb/March — model expects increasing seasonal activity heading into Q2.
- Smooth transition from real to forecasted data — no sudden jumps.
- Note: Recursive forecasting works well for 3-month horizons. Beyond 6 months, error accumulation reduces reliability.

6. Project Summary — Key Findings

Category	Finding	Detail
Top Restaurant	Bob's Diner	~94% of all sales in the dataset
Busiest Day	Friday	Highest sales across all 3 years
Quietest Day	Sunday	Customers prefer home meals at weekend end
Peak Season	Q2 (Apr – Jun)	Consistently the strongest quarter
Weakest Season	Q4 (Oct – Dec)	Consistently weakest — holiday boost does not apply
Most Popular Item	Strawberry Smoothy	Over 236,000 units sold — top by large margin
Volume vs Revenue	Aligned at Bob's Diner	Highest volume = highest daily revenue
Most Expensive Item	Fou Cher — $53.98	Premium fine-dining positioning

Model Results Summary

Model	Metric	Result	Verdict
Linear Regression	RMSE	~230	Poor — cannot capture non-linear patterns
Random Forest	RMSE	~60 (Best)	Strong — learns feature interactions
XGBoost	RMSE	~62	Close second — strong gradient boosting
LSTM (Deep Learning)	MAPE	10.55% (Good)	Captures temporal patterns — suitable for business planning

7. Interview Questions and Answers

These questions cover the key concepts from this project and are commonly asked in data science and ML engineering interviews.

Data Preprocessing

Q: Why did you filter out zero rows before running IQR outlier detection?

If we include zero-sales rows, Q1 = Q3 = 0, so IQR = 0. Every non-zero value would be flagged as an outlier, which is incorrect.

By filtering to rows where item_count > 0 first, the IQR calculation reflects the real distribution of actual sales values.

This is a common preprocessing step when a dataset has structural zeros (days when an item simply wasn't sold).

Q: What is data leakage and how did you prevent it in this project?

Data leakage occurs when information from the future (test set) influences the training process, making the model appear more accurate than it really is.

In this project we prevented leakage in two ways:

1. Time-based train/test split — we never let the model see future dates during training.
2. Fitting MinMaxScaler only on training data — if we fitted on the entire dataset, the scaler would encode the range of future values, leaking information about the test set.

Q: Why did you merge three CSV files and what join type did you use?

The information needed for analysis was spread across three tables. By merging, we create one row with full context: date, item name, restaurant name, count sold, price, and calories.

We used left joins with sales.csv as the base: sales → items on item_id, then → restaurants on store_id.

No rows were lost in the merge, which confirmed all IDs matched correctly across all three files.

Exploratory Data Analysis

Q: What was the most important finding in your EDA and how did it influence your modeling?

The most important finding was the strong weekly cycle — Friday is consistently the highest sales day, Sunday the lowest, across all three years.

This directly influenced feature engineering: we extracted dayofweek as a numeric feature for ML models.

The EDA finding was validated post-modeling: dayofweek scored the highest feature importance (~0.53) in both Random Forest and XGBoost.

This alignment between EDA and feature importance is a sign the models learned real structure, not noise.

Q: Bob's Diner accounts for 94% of sales. How does this affect your analysis?

It means the aggregate dataset patterns are essentially Bob's Diner's patterns.

Any trend we find — weekly cycles, seasonal peaks, day-of-week effects — is primarily driven by one restaurant.

This is important context when presenting findings to stakeholders: improvements to Bob's Diner operations will have a disproportionate impact on the overall business.

For more granular analysis, we would ideally build separate models per restaurant.

Machine Learning Models

Q: Why did Linear Regression perform so poorly compared to Random Forest?

Linear Regression fits a single straight line through the feature space. It assumes the relationship between features and target is linear.

But sales are driven by interactions: being Friday AND being in summer produces a multiplicative spike — not just the sum of Friday effect plus summer effect.

Random Forest builds decision trees that can capture these non-linear interactions: 'IF dayofweek = Friday AND month = June THEN predict high sales'.

This is why Random Forest RMSE (~60) was roughly 4x lower than Linear Regression RMSE (~230).

Q: What is the difference between bagging (Random Forest) and boosting (XGBoost)?

Bagging (Random Forest): Builds many independent trees in parallel, each on a random sample of data and features. Final prediction is the average. Reduces variance.

Boosting (XGBoost): Builds trees sequentially. Each new tree specifically focuses on correcting the errors made by the previous trees. Reduces bias.

In this project, both methods performed similarly (~60 vs ~62 RMSE), which is common when the dataset is clean and the seasonal patterns are strong.

XGBoost tends to win on larger, messier tabular datasets where iterative error correction is more valuable.

Q: Why did you use a time-based train/test split instead of random split?

For time series data, random splitting is invalid. A random split would let the model train on data from 2021 and test on data from 2019 — it would be 'predicting the past' rather than the future.

More critically, random splitting allows the model to see future values during training, which constitutes data leakage.

Time-based split mirrors the real-world deployment scenario: the model is trained on all historical data and must predict what comes next.

LSTM and Deep Learning

Q: What is an LSTM and why is it suited for time series forecasting?

LSTM stands for Long Short-Term Memory. It is a type of Recurrent Neural Network (RNN) designed to retain information across long sequences.

Standard neural networks treat each input independently — they have no memory of previous inputs.

LSTM has a memory cell with three gates: forget gate (decide what to discard), input gate (decide what to store), output gate (decide what to output).

For sales forecasting, this means the model can 'remember' that last week was high season and use that context to predict today's sales — something ML models with engineered features cannot do as naturally.

Q: What is the sliding window technique and why did you use 30 days?

The sliding window creates sequences from time series data for LSTM input.

We feed the model 30 consecutive days as input, and it predicts the 31st day.

The window then slides forward by one day: days 2-31 predict day 32, and so on.

30 days was chosen because it captures one full monthly cycle — enough seasonal context without making sequences too long or training too slow.

This technique transforms the raw time series into thousands of (input sequence, target) training pairs.

Q: Why did you use EarlyStopping and what does patience=10 mean?

Without EarlyStopping, the model would train for all 50 epochs even if it stopped improving — or started getting worse — after epoch 20.

Continuing past the optimal point causes overfitting: the model memorises training patterns and performs worse on new data.

patience=10 means: if the validation loss does not improve for 10 consecutive epochs, stop training and restore the best weights.

This automatically finds the optimal training point without manual tuning.

Q: What is MAPE and what does your LSTM result of 10.55% mean?

MAPE = Mean Absolute Percentage Error. It expresses the average prediction error as a

percentage of the actual value.

10.55% means the model's predictions are off by about 10.55% of actual sales on average.

The scale: below 10% = Excellent, 10-20% = Good, above 20% = Needs improvement.

For this dataset with daily sales swinging from $2,000 to $12,000 (highly volatile), 10.55% is practically strong and sufficient for business planning like staffing and inventory decisions.

Q: What is recursive forecasting and what are its limitations?

Recursive (auto-regressive) forecasting: predict Day 1, then feed that prediction as input to predict Day 2, and so on.

We use it because real future values do not exist yet — we cannot feed the model actual future data.

Limitation: errors accumulate over the forecast horizon. A small error in Day 1 prediction affects Day 2, which affects Day 3, etc.

For 3-month (90-day) forecasts, the model performs well. Beyond 6 months, accumulated error reduces reliability.

Alternative approaches: direct multi-step forecasting (train one model per forecast horizon) avoids error accumulation but requires more models.

Conceptual and Industry Questions

Q: When would you choose LSTM over Random Forest for forecasting?

Choose LSTM when: the sequence order matters critically, you have enough data (ideally 2+ years of daily data), you want the model to learn temporal dependencies automatically without feature engineering, and you can tolerate longer training times.

Choose Random Forest when: you have limited data, need interpretability (feature importance), want fast training, or the temporal dependencies are already captured by engineered features like dayofweek and month.

In this project both approaches worked — Random Forest won on RMSE, LSTM was evaluated on MAPE for sequence prediction. In production, Random Forest would be faster to retrain and explain.

Q: How would you improve this model for production deployment?

Add external features: weather data (temperature affects restaurant visits), local events, holidays, and promotional calendar.

Build per-restaurant models instead of one aggregate model — especially given Bob's Diner dominates the data.

Use cross-validation designed for time series: TimeSeriesSplit with multiple rolling windows.

Add prediction intervals (confidence bands), not just point forecasts — business decisions need to understand the uncertainty range.

Automate retraining when new data arrives and monitor for model drift (when real-world patterns

change from training patterns).

Q: What does it mean for a model to overfit, and how did you prevent it?

Overfitting: the model memorises the training data including its noise, so it performs well on training data but poorly on unseen test data.

In Random Forest: overfitting is controlled by the ensemble averaging (many trees smooth out individual errors) and the random feature subsampling.

In LSTM: we used two regularisation techniques — Dropout (randomly deactivates 20% of neurons each training step) and EarlyStopping (stops training before overfitting begins).

The fact that train and validation loss curves tracked closely together in the LSTM training plot confirms no significant overfitting occurred.

---

8. Quick Reference — Key Terms

Term	Plain English Definition
RMSE	Root Mean Squared Error — average prediction error in same units as target. Lower is better.
MAPE	Mean Absolute Percentage Error — average error as a % of actual value. Below 10% = Excellent, 10-20% = Good.
IQR	Interquartile Range = Q3 − Q1. Used to detect statistical outliers (values beyond 1.5×IQR from Q1/Q3).
Feature Engineering	Creating new numeric input variables from raw data (e.g. extracting month from a date string).
Data Leakage	When future information accidentally influences model training, causing inflated accuracy.
Bagging	Build many independent models and average their predictions (Random Forest uses this).
Boosting	Build models sequentially, each correcting the errors of the previous one (XGBoost uses this).
LSTM	Long Short-Term Memory — a neural network that retains memory across time steps, suited for sequences.
Sliding Window	Technique that creates (sequence, target) pairs from time series for LSTM training.
MinMaxScaler	Scales values to [0,1] range. Must be fit on training data only to avoid leakage.
Dropout	Regularisation technique that randomly deactivates neurons during training to prevent overfitting.

EarlyStopping	Stops training when validation loss stops improving, preventing overfitting.
Recursive Forecast	Use each prediction as input for the next — allows forecasting many steps without real future data.
Feature Importance	Score showing how much each input feature contributed to the model's predictions.